

IMPLEMENTATION_PLAYBOOK_PLAIN

How to Build Helios, Step by Step — In Plain English

Plain-language companion to `docs/planning/IMPLEMENTATION_PLAYBOOK.md`. For the exact spec, read that / its PDF.

In one sentence

This is the build manual: it tells you, milestone by milestone (**M0** through **M12**), exactly what files to create, what commands to run, what tests must pass, and what mistakes will quietly ruin your data if you're careless.

Why this matters to you

The playbook is the "do these things in this order" guide. The actual code already lives in other docs (the full dbt SQL, the server skeletons, the metric registry). Your job is **assembling and testing**, not inventing. This companion makes the sequence feel doable: build each milestone, pass its gate, move on. Skip ahead and downstream numbers go wrong *without any error message* — that's the trap this manual exists to prevent.

The big ideas, simply

- **Build in order. Each milestone has an "exit gate."** A gate is one concrete check (a test passes, a query returns rows). Don't move on until it's green.
- **You're copying code, not writing it.** Every file says "copy from `DBT_GUIDE.md` section X." Files marked "already exists" must NOT be regenerated.
- **Three "levels" of done. L1** = working basic loop (after M7). **L2** = full system + benchmark proof (after M10). **L3** = autonomous, scheduled, deep (after M11). You can stop at any level and still have something valuable.

- **Some bugs fail silently.** A wrong session key still looks like a key. A broken funnel flag still produces a number. That's why M3 (the keystone) demands you write tests *first*, then make them pass.
- **The grounding rules are enforced by structure, not trust.** The LLM physically cannot write SQL or do math — it can only call tools. The tools refuse anything off-script.

What you actually do (in order)

Each milestone below = one chunk of work with a one-line "you're done when" gate.

- **M0 — Foundation.** Set up the repo, Python environment, Google Cloud login, and dbt config. *Done when:* `dbt debug` says all green and a small test query returns rows cheaply.
- **M1 — Sources, macros, seed.** Declare where the GA4 data lives; create the four shared helpers (param extractor, session-key builder, the one channel-grouping rule, the revenue-check test). *Done when:* `dbt deps` + `dbt seed` succeed; macros compile.
- **M2 — Staging.** Two "rename-and-clean" models that copy raw GA4 1:1 into tidy columns. No joins, no math. *Done when:* `dbt build --select staging` passes its tests.
- **M3 — Sessionization + funnel (THE KEYSTONE).** Rebuild the "session" and the "funnel" that GA4 never ships as rows. **Write the golden tests first** — these fail silently if wrong. *Done when:* the session key is unique and the funnel is monotonic (each step $\leq$ the one before).
- **M4 — Marts.** The wide, ready-to-query tables the rest of the system reads. *Done when:* `dbt build` is green, revenue reconciles **to the cent**, and channels = exactly **10**.
- **M5 — Semantic layer live.** The metric registry already exists; you add a validator that proves every metric reference resolves. *Done when:* `validate_semantic.py` prints PASS with 0 dangling references.
- **M6 — Grounding MCP pair.** Build `semantic-mcp` (the only path to SQL) and `warehouse-mcp` (the only thing that talks to BigQuery, with a dry-run + budget gate). *Done when:* a query round-trips and reconciles within 0.5%.
- **M6b — Stats / experiment / report MCP.** Build the math servers. The star is `decompose_change` (the mix-vs-rate split). *Done when:* its golden test passes exactly (`mix=-0.0018, rate=0`).
- **M7 — Minimal loop (L1 DONE).** A simple Plan → Monitor → Narrate run that finds one anomaly and writes a brief. *Done when:* one anomaly → a brief in under 5 minutes, with zero made-up columns.
- **M8 — Memory.** Tables so Helios remembers across runs; seed the seasonality calendar (Black Friday, December, January). *Done when:* save-then-recall round-trips.
- **M9 — Full 7-agent loop.** Add the other five agents and the Critic that attacks each finding. *Done when:* findings carry significance + a dollar figure + a recommended experiment.

- **M10 — Eval + CI (L2 DONE).** Prove the headline claim: **$\geq 85\%$ root-cause accuracy vs $\leq 45\%$ for a naive baseline**, with zero hallucination, wired into CI. *Done when:* the gate is green.
- **M11 — Autonomy & depth (L3 DONE).** Run on a schedule; add forecasting, cohorts, the full Critic battery. *Done when:* scheduled runs finish in <5 min and accuracy holds across every dimension.
- **M12 — Productionization & frontier (deferred).** Documented roadmap only — multi-tenant, causal inference. Not built on this dataset, and the doc explains honestly why.

Easy things to get wrong

- **Skipping M3's golden tests** because the structural tests pass. Shape checks (not-null, unique) never catch wrong *math*. Write the golden tests first.
- **The traffic_source first-touch leak** (the single highest-value bug). GA4's `traffic_source.*` is the user's *first-ever* channel, identical on every session they have. Use session-scoped source/medium; fall back to `traffic_source` only when null.
- **Inventing an 11th channel group** (like "Paid Other"). There are exactly **10**, defined in one macro. Anything else is a hard error.
- **Storing rates instead of counts** in `fct_daily_funnel`. Store raw counts; compute rates later as `SUM(numerator)/SUM(denominator)`. Pre-dividing reintroduces Simpson's paradox.
- **Not deduping revenue per transaction_id**. GA4 emits duplicate purchase rows; failing to collapse them roughly doubles revenue and AOV.
- **Running dbt run then dbt test separately**. Always `dbt build` — it interleaves build and test so a poisoned upstream aborts before reaching the marts.
- **Letting the LLM control flow or write SQL**. The runner is plain Python; the model only fills in tool calls. Enforce the per-agent tool allow-list structurally, not by asking nicely in a prompt.
- **Grading on live data**. Real anomalies have no known cause, so you can't score accuracy. Always grade against the *injected* labels, and make hallucination a hard-zero gate.

Glossary — the exact words, demystified

- **Milestone (M0–M12):** one numbered chunk of the build, each with its own exit gate.
- **Exit gate:** the single concrete check that says a milestone is truly finished.
- **L1 / L2 / L3:** the three "levels of done" — minimal loop, full benchmarked system, autonomous deep system.
- **Keystone (M3):** the load-bearing transform; if it's wrong, everything downstream is silently wrong.

- **Golden test:** a test with hand-worked correct numbers (e.g. `mix=-0.0018`) that catches wrong math, not just wrong shape.
- **Monotonic funnel:** each step's count is $\leq$ the step before, so conversion rates can never exceed 1.
- **reached_* flags:** "did this session reach this step *or any later one*" — the trick that keeps the funnel monotonic. (The old `did_*` flags are retired.)
- **Reconcile:** check that your computed total matches the raw source total; >0.5% drift fails (orders must match to the cent).
- **MCP server:** a tool service the LLM calls. `semantic-mcp` = only path to SQL; `stats-mcp` = only path to math; `warehouse-mcp` = only thing that runs queries.
- **dry_run:** a cost preview of a query before running it; required before every real run, capped at the byte budget.
- **Byte budget:** the 5 GiB-per-run scan cap — the main cost guardrail.
- **Critic:** the agent whose job is to *attack* each finding (mix-shift, small sample, seasonality, data quality) before it ships.
- **Naive baseline:** the dumb "biggest segment delta" method Helios must beat (~45% vs $\geq 85\%$).
- **Faithfulness:** every number in the brief traces back to an actual tool output, never invented by the model.

When to open the real doc

Open `pdf/docs/planning/IMPLEMENTATION_PLAYBOOK.pdf` (or the `.md`) when you need the exact files-to-copy table, the precise commands, the full test lists, or the troubleshooting appendix for a specific milestone. This companion gives you the map; the real doc gives you the turn-by-turn directions.